



CANADIAN
INTERNATIONAL COLLEGE

THE FUTURE IS YOURS

Stock Market Analysis System

Project Documentation

Supervised by

DR. Mohamed Khalaf

Prepared by

Name	Student ID	Role
Ahmed Elsayed	202206708	Tester / Documentation
Ahmed Atef	202206546	Backend Developer
Mohamed Hossam	202206596	Data Processing & Analysis
Malak Sherif	202206715	Visualization Developer
Nagat Shahin	202206566	UI Developer

1. Project Overview

The **Stock Market Analysis System** is a web-based application designed to provide users with real-time stock market analysis, historical data visualization, and technical analysis tools. The system fetches live stock information from Yahoo Finance and displays interactive charts with moving averages to help investors and traders make informed decisions.

The application is built using **Streamlit** as the frontend framework, **yfinance** for data acquisition, **pandas** for data processing, and **Plotly** for interactive visualizations. It provides an intuitive interface for searching stocks, viewing current market prices, and analyzing historical price trends.

2. Objectives

- **Provide Real-Time Stock Information:** Fetch and display current stock prices, market data, and company information from Yahoo Finance.
- **Enable Historical Analysis:** Allow users to view historical price and volume data across multiple time periods (7 days, 1 month, 3 months, 6 months, 1 year, 5 years).
- **Visual Analytics:** Generate interactive charts displaying price trends and trading volumes with technical indicators (moving averages).
- **User-Friendly Interface:** Provide an intuitive sidebar interface with preset popular stock symbols and custom symbol search capabilities.
- **Data Validation:** Ensure accurate data handling with robust input validation and error management.
- **Performance Optimization:** Implement caching mechanisms to reduce API calls and improve application responsiveness.

3. Scope

In Scope:

- Real-time stock data fetching from Yahoo Finance API
- Display of current stock information (price, market cap, volume, exchange info)
- Historical price and volume charts
- Moving average technical indicator calculation and visualization
- Support for multiple time periods (7d, 1mo, 3mo, 6mo, 1y, 5y)
- Popular stock symbols (AAPL, MSFT, TSLA, NVDA, BTC-USD, ^GSPC)
- Custom symbol search with suggestions
- Responsive web interface using Streamlit
- Data caching to optimize performance

Out of Scope:

- Real-time stock alerts/notifications
- Portfolio management features
- User authentication and account management
- Advanced technical indicators (beyond moving averages)
- Machine learning-based stock predictions
- Trading execution capabilities
- Mobile application
- Historical data storage/database

4. Functional Requirements

FR#	Requirement	Description
FR1	Search Stock Symbol	Users can select from popular symbols or enter a custom stock symbol to analyze.
FR2	Fetch Stock Information	System retrieves current stock price, company name, exchange, currency, market cap, and volume.
FR3	Historical Data Retrieval	System fetches historical price and volume data for the selected time period.
FR4	Moving Average Calculation	System calculates and displays moving averages based on user-specified window size (2-60 days).
FR5	Price Trend Chart	Display interactive price trend chart showing open, close, high, low prices and moving average.
FR6	Volume Chart	Display interactive volume chart showing trading volume over time.
FR7	Price Change Indicator	Calculate and display price change percentage and direction.
FR8	Symbol Suggestions	Provide auto-suggestions when user enters potentially invalid or misspelled symbols.
FR9	Multiple Time Periods	Support analysis across 7 different historical time periods.
FR10	Data Formatting	Format currency values, large numbers, and dates appropriately for display.
FR11	Error Handling	Display user-friendly error messages when API calls fail or invalid symbols are provided.
FR12	Sidebar Controls	Provide sidebar interface for symbol selection, period selection, and moving average configuration.

Non-Functional Requirements

NFR#	Requirement	Description
NFR1	Performance	API data should be cached for 5 minutes to reduce redundant calls and improve response time.
NFR2	Availability	System should maintain 99% uptime (dependent on Yahoo Finance API availability).
NFR3	Responsiveness	Charts should render within 3 seconds of user interaction.
NFR4	Scalability	Support concurrent users without performance degradation.
NFR5	Security	No user data storage; all data is stateless and processed in real-time.
NFR6	Compatibility	Support Python 3.11+ and major browsers (Chrome, Firefox, Edge, Safari).
NFR7	Error Recovery	Graceful error handling with informative messages and recovery suggestions.
NFR10	Data Accuracy	Ensure data accuracy by validating API responses and handling edge cases.
NFR9	Code Quality	Type hints, docstrings, and validation throughout the codebase.

5. Tools and Technologies

Component	Technology	Version	Purpose
Frontend	Streamlit	1.36.0+	Web framework for building interactive UI
Backend	Python	3.11+	Primary programming language
Data API	yfinance	0.2.40+	Yahoo Finance data fetching library
Data Processing	pandas	2.2.0+	Data manipulation and analysis
Charting	Plotly	5.22.0+	Interactive visualization library
Development	VS Code	Latest	Code editor and IDE
Version Control	Git	Latest	Code repository management

Architecture Layers:

- **Presentation Layer:** Streamlit UI with interactive components
- **Business Logic Layer:** Data processing and calculations (data_processing.py)

- **Data Access Layer:** Stock API integration (stock_api.py)
- **Validation Layer:** Input validation and error handling (validation.py)
- **Visualization Layer:** Chart generation and formatting (charts.py)

6. System Workflow

Main User Flow:

- **User Launches Application** → Streamlit loads the main interface
- **User Selects Symbol** → Choose from popular symbols or enter custom symbol
- **User Configures Parameters** → Select time period and moving average window
- **User Initiates Analysis** → Click "Analyze Stock" button
- **System Validates Input** → Symbol validation and normalization
- **System Fetches Data** → Retrieve stock info and historical data from Yahoo Finance
- **System Processes Data** → Calculate moving averages and price changes
- **System Renders Charts** → Display interactive price trend and volume charts
- **System Displays Information** → Show current stock metrics and statistics
- **User Interacts with Charts** → Hover, zoom, and interact with visualizations

Data Processing Workflow:

1. Fetch raw OHLCV (Open, High, Low, Close, Volume) data from yfinance
2. Normalize and format the historical data
3. Calculate moving averages for trend analysis
4. Calculate price changes and percentage changes
5. Format currency and large numbers for display
6. Prepare data for Plotly visualization

Error Handling Workflow:

1. Validate stock symbol format and existence
2. Check API response validity
3. Handle missing or incomplete data

4. Provide user-friendly error messages
5. Suggest corrections or alternatives

7. User Interface Requirements

Layout:

Main Heading: "Stock Market Analysis System" with descriptive caption

Sidebar Panel (Left):

"Search Settings" section

Popular Symbols dropdown with formatted display

Custom Symbol text input with placeholder

Historical Period selection

Moving Average Window slider (2-60 range)

"Analyze Stock" primary button

Main Content Area (Center/Right):

Stock information cards (Company name, currency, exchange)

Current price and price change indicator

Market metrics (Market Cap, Volume, Day High/Low)

Interactive price trend chart with moving average

Interactive volume chart

Symbol suggestions when applicable

UI Components:

Sidebar Selectbox: For popular symbols and time periods

Text Input: For custom stock symbol entry

Slider: For moving average window selection

Button: Primary action button for stock analysis

Metric Cards: Display key stock metrics

Charts: Interactive Plotly charts for price and volume

Info/Error Messages: Display suggestions, info, or error states

Design Principles:

Clean, professional appearance

Intuitive navigation and user guidance

Responsive layout that adapts to screen size

Clear visual hierarchy

Accessible and user-friendly

8. Testing Requirements

Unit Testing:

Validation Module Tests:

Test symbol normalization

Test period validation

Test moving average window validation

Test symbol suggestions

Data Processing Tests:

Test moving average calculation accuracy

Test currency formatting

Test date formatting

Test price change calculations

Stock API Tests:

Test fetch_stock_info() with valid/invalid symbols

Test fetch_historical_data() for various periods

Test error handling for API failures

Chart Generation Tests:

Test price trend chart creation

Test volume chart creation

Test chart rendering with various data sets

Integration Testing:

End-to-end workflow testing (symbol selection → data fetch → chart display)

API integration with Yahoo Finance

Data pipeline integration (fetch → process → display)

Error handling across components

Performance Testing:

Cache effectiveness (verify 5-minute TTL)

Chart rendering performance

API response time

Memory usage with large datasets

User Acceptance Testing:

UI usability and intuitiveness

Data accuracy verification

Symbol search accuracy

Error message clarity

Chart interactivity

Test Data:

Valid symbols: AAPL, MSFT, TSLA, NVDA, BTC-USD, ^GSPC

Invalid symbols: INVALID123, XYZ999, !!!

Various time periods: 7d, 1mo, 3mo, 6mo, 1y, 5y

Edge cases: Symbols with special characters, very new symbols, delisted symbols

9. Expected Output

Output 1: Stock Information Display

Company Name and Symbol

Current Price with Currency

Exchange and Market Information

Price Change (percentage and direction)



Output 2: Market Metrics

Market Capitalization (formatted as billions/millions)

Trading Volume

Day High and Day Low prices

Open and Previous Close prices



Output 3: Price Trend Chart

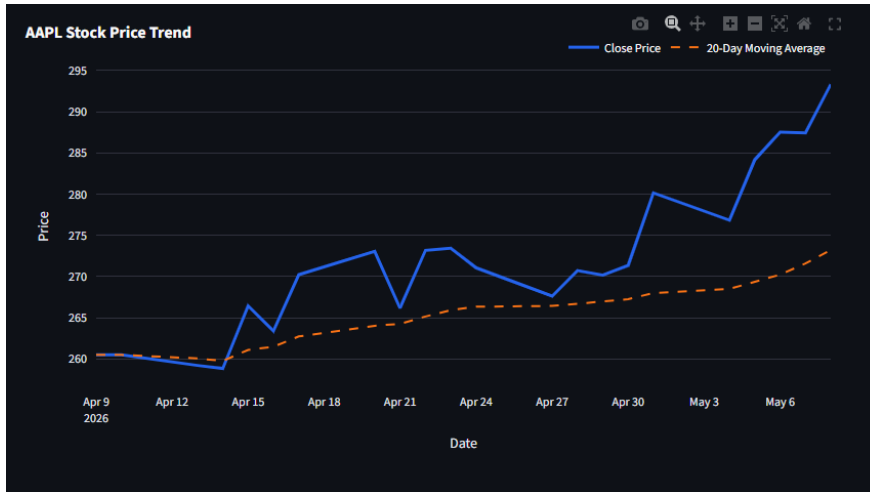
X-axis: Date/Time

Y-axis: Price (\$)

Lines: Open, Close, High, Low, Moving Average

Interactive: Hover for details, zoom, pan capabilities

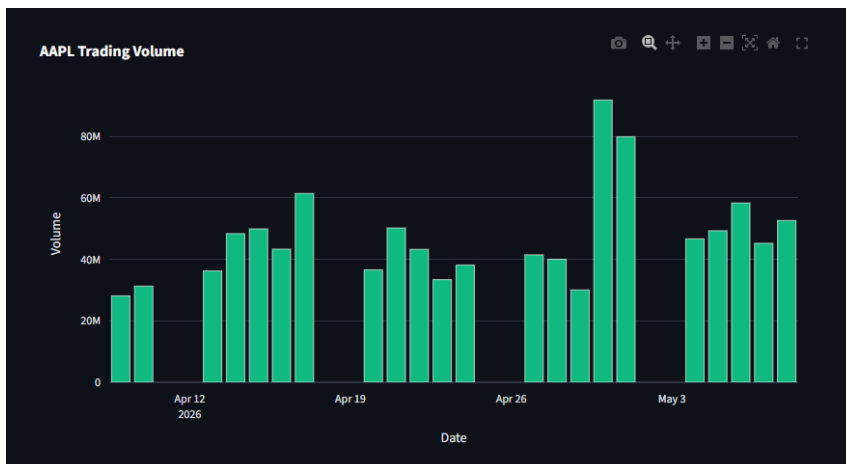
Legend showing all data series



Output 4: Volume Chart

X-axis: Date/Time Bars: Daily volume

Interactive: Hover for volume details, zoom, pan capabilities

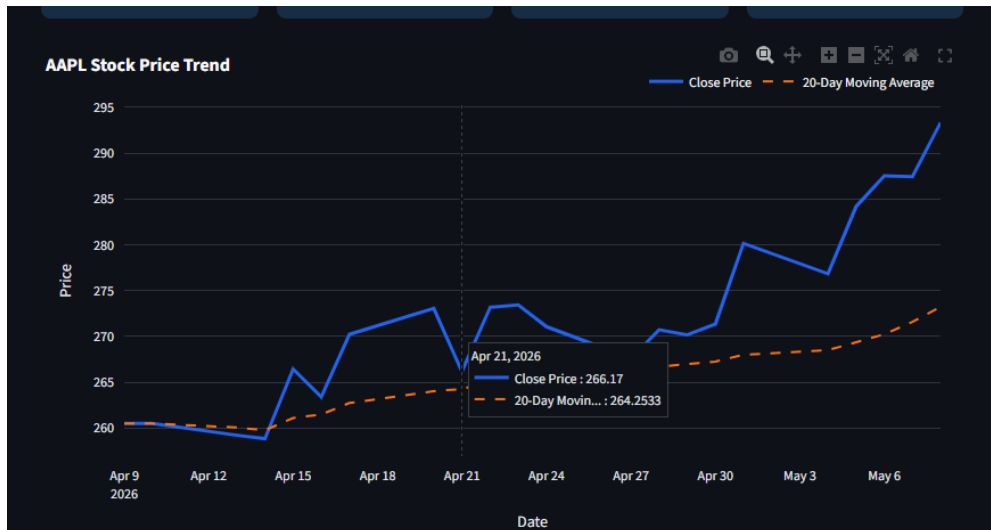


Output 5: Technical Analysis

Moving Average overlay on price chart

Moving average values at each data point

Price-to-Moving Average relationship visualization



Output 6: Error Messages and Suggestions

Clear error messages for invalid symbols

Suggested symbols for user corrections

Informative messages for common symbols

Search Settings

Popular Symbols
Custom symbol

Stock Symbol
Example: AAPL, TSLA, MSFT, BTC-US

You can type a Yahoo Finance symbol or choose one above.

Historical Period
1 Month

Moving Average Window
20

Analyze Stock

Stock Market Analysis System

Backend-powered stock lookup using Yahoo Finance, pandas, and Plotly.

Please enter a stock symbol.

Try a common symbol such as AAPL, MSFT, TSLA, NVDA, BTC-USD, or ^GSPC.

Output 7: Historical stock Data

Historical Stock Data

Date	Open	High	Low	Close	Volume	MA_20
2026-05-08 00:00:00	416.48	431.2	416.39	428.35	64880800	386.7615
2026-05-07 00:00:00	407.48	415.83	402.12	411.79	64294200	382.7915
2026-05-06 00:00:00	386.25	401.68	384.02	398.73	53465400	379.483
2026-05-05 00:00:00	395.19	402.12	389	389.37	47780600	378.47
2026-05-04 00:00:00	390.23	394.64	384.8	392.51	48765100	377.8644
2026-05-01 00:00:00	382.49	397.82	378.8	390.82	65338300	377.0029
2026-04-30 00:00:00	372.75	384.75	368.17	381.63	51076000	376.1394
2026-04-29 00:00:00	375.4	376.4	370.04	372.8	45384800	375.7733
2026-04-28 00:00:00	374.68	382.29	372.54	376.02	50864400	375.9857
2026-04-27 00:00:00	372.09	380.78	364.02	378.67	66735800	375.9831

10. Conclusion

The **Stock Market Analysis System** is a comprehensive web-based application designed to democratize stock market analysis. By leveraging real-time data from Yahoo Finance and interactive visualizations powered by Plotly, it enables both novice and experienced investors to make informed investment decisions.

The system successfully combines simplicity with functionality, providing an intuitive user interface that doesn't sacrifice analytical capability. The modular architecture ensures maintainability and scalability for future enhancements.

Key Achievements:

- Real-time access to global stock market data
- User-friendly interface for quick analysis
- Professional-grade interactive charts
- Robust error handling and validation
- Optimized performance through intelligent caching

Future Enhancements:

- Advanced technical indicators (RSI, MACD, Bollinger Bands)
- Portfolio tracking and comparison
- Price alert notifications
- Export reports (PDF, CSV)
- User preferences and watchlists
- Machine learning-based sentiment analysis
- Mobile application

Real-time streaming data

Maintenance and Support:

Regular updates to dependencies

Monitoring API availability and changes

User feedback collection and implementation

Performance optimization

Security updates and patch